

```

const express = require("express");
const Anthropic = require("@anthropic-ai/sdk");
const cors = require("cors");
const rateLimit = require("express-rate-limit");

const app = express();
const anthropic = new Anthropic({ apiKey: process.env.ANTHROPIC_API_KEY });

app.use(cors());
app.use(express.json({ limit: "50mb" }));

const limiter = rateLimit({ windowMs: 60 * 60 * 1000, max: 20 });
app.use("/api/", limiter);

const users = {};
const cache = {};

function getUser(userId) {
  if (!users[userId]) users[userId] = { uses: 0, subscribed: false, quizHistory:
  return users[userId];
}

function cacheKey(str) {
  let hash = 0;
  for (let i = 0; i < Math.min(str.length, 500); i++) {
    hash = ((hash << 5) - hash) + str.charCodeAt(i);
    hash |= 0;
  }
  return hash.toString();
}

function buildContentParts(text, images) {
  const parts = [];
  if (images && images.length > 0) {
    for (const img of images) {
      if (img.isPdf) continue;
      parts.push({ type: "image", source: { type: "base64", media_type: img.mimeType
    }
  }
  if (text) parts.push({ type: "text", text: `=== TEST CONTENT ===\n${text}\n===
  return parts;
}

function parseAnswers(raw) {

```

```

const lines = raw.split("\n");
const answers = [];
for (const line of lines) {
  const m = line.trim().match(/^Q(\d+):\s*(.+)/i);
  if (m) answers.push({ q: parseInt(m[1]), a: m[2].trim(), conf: 95 });
}
if (answers.length === 0) {
  for (const line of lines) {
    const m = line.trim().match(/^(\d+)[.]:\s+(.+)/);
    if (m) answers.push({ q: parseInt(m[1]), a: m[2].trim(), conf: 88 });
  }
}
return answers;
}

// HEALTH
app.get("/", (req, res) => res.json({ status: "Test Taker API running", modes: ["

// USAGE
app.get("/api/usage/:userId", (req, res) => {
  const user = getUser(req.params.userId);
  res.json({ uses: user.uses, freeUsesLeft: Math.max(0, 3 - user.uses), subscribe
});

// MODE 1A - MAP (Claude Sonnet - Tier 3)
app.post("/api/map", async (req, res) => {
  try {
    const { userId, subject, text, images } = req.body;
    if (!userId) return res.status(400).json({ error: "userId required" });
    const user = getUser(userId);
    if (!user.subscribed && user.uses >= 3) return res.status(402).json({ error:

    const key = "map_" + cacheKey((text || "") + subject);
    if (cache[key]) return res.json({ ...cache[key], cached: true });

    const contentParts = buildContentParts(text, images);
    contentParts.push({ type: "text", text: `You are an expert test structure ana

```

Your job is ONLY to identify and map relationships. Do NOT answer questions. Do N

STEP 1 - Find all components:

- Questions numbered Q1, Q2, Q3...
- Passages: blocks of reading text (label Passage A, B, C...)
- Images/Diagrams (label Image 1, 2, 3...)
- Question groups sharing context

STEP 2 - For each question determine:

- Which passage it references (explicit mention OR layout position OR topic match)
- Which image it references
- Whether it is standalone

STEP 3 – Output ONLY this JSON array:

```
[
  {"question":1,"linked_passage":"A","linked_image":null,"standalone":false,"confid
  {"question":2,"linked_passage":null,"linked_image":1,"standalone":false,"confid
  {"question":3,"linked_passage":null,"linked_image":null,"standalone":true,"conf
]
```

Include ALL questions. Use honest confidence scores. Output ONLY the JSON array.`

```
const message = await anthropic.messages.create({
  model: "claude-sonnet-4-20250514",
  max_tokens: 4000,
  messages: [{ role: "user", content: contentParts }],
});

const raw = message.content.map(b => b.text || "").join("").trim();
let mappings = [];
try {
  const match = raw.match(/\[[\s\S]*\]/);
  if (match) mappings = JSON.parse(match[0]);
} catch (e) {
  return res.status(500).json({ error: "Could not parse mapping", raw: raw.su
}

if (!user.subscribed) users[userId].uses += 1;
const result = { mappings, usesLeft: Math.max(0, 3 - users[userId].uses) };
cache[key] = result;
res.json(result);
} catch (err) {
  res.status(500).json({ error: err.message });
}
});

// MODE 1B – SOLVE (Claude Sonnet – Tier 3)
app.post("/api/solve", async (req, res) => {
  try {
    const { userId, subject, text, images, mappings } = req.body;
    if (!userId) return res.status(400).json({ error: "userId required" });
    const user = getUser(userId);
    if (!user.subscribed && user.uses >= 3) return res.status(402).json({ error:

    const contentParts = buildContentParts(text, images);
    const mappingContext = mappings && mappings.length > 0
```

```
? `\\nKnown structure:\\n${mappings.map(m => `Q${m.question}: ${m.linked_pass
: "";
```

```
contentParts.push({ type: "text", text: `You are an expert ${subject} test so
${mappingContext}
```

1. Read the ENTIRE test first
2. For each question use ONLY its linked passage/image to answer
3. Never mix up which passage belongs to which question

Answer every question in this format:

Q1: C

Q2: B - Photosynthesis converts sunlight into glucose

Q3: 1865

Start each answer with Q<number>: then the answer. No intro, no summary. Answer e

```
const message = await anthropic.messages.create({
  model: "claude-sonnet-4-20250514",
  max_tokens: 8000,
  messages: [{ role: "user", content: contentParts }],
});

const raw = message.content.map(b => b.text || "").join("").trim();
const answers = parseAnswers(raw);
if (answers.length === 0) return res.status(500).json({ error: "Could not par

if (!user.subscribed) users[userId].uses += 1;
res.json({ answers, usesLeft: Math.max(0, 3 - users[userId].uses) });
} catch (err) {
  res.status(500).json({ error: err.message });
}
});
```

// MODE 2A - GENERATE QUIZ (Claude Haiku - Tier 2, cheaper)

```
app.post("/api/quiz/generate", async (req, res) => {
  try {
    const { userId, subject, text, gradeLevel } = req.body;
    if (!userId) return res.status(400).json({ error: "userId required" });
    const user = getUser(userId);
    if (!user.subscribed && user.uses >= 3) return res.status(402).json({ error:

    const key = "quiz_" + cacheKey((text || "") + subject + (gradeLevel || ""));
    if (cache[key]) return res.json({ ...cache[key], cached: true });

    const message = await anthropic.messages.create({
      model: "claude-haiku-4-5-20251001",
      max_tokens: 3000,
```

```
messages: [{ role: "user", content: `You are a ${subject} teacher for ${gra
```

Generate a 10-question diagnostic quiz based on this material.

Mix: 30% easy, 40% medium, 30% hard.

Cover all major topics. Do NOT copy questions from the original – create new ones

Material:

```
${(text || "").substring(0, 15000)}
```

Output ONLY this JSON:

```
{"topics":["Topic 1","Topic 2"],"quiz":[{"id":1,"question":"What is...?","options
```

Output ONLY the JSON, nothing else.` }],

```
});
```

```
const raw = message.content.map(b => b.text || "").join("").trim();
```

```
let quizData = null;
```

```
try {
```

```
  const clean = raw.replace(/```json|```/gi, "").trim();
```

```
  quizData = JSON.parse(clean);
```

```
} catch (e) {
```

```
  return res.status(500).json({ error: "Could not parse quiz", raw: raw.subst
```

```
}
```

```
if (!user.subscribed) users[userId].uses += 1;
```

```
const result = { quizData, usesLeft: Math.max(0, 3 - users[userId].uses) };
```

```
cache[key] = result;
```

```
res.json(result);
```

```
} catch (err) {
```

```
  res.status(500).json({ error: err.message });
```

```
}
```

```
});
```

```
// MODE 2B – ANALYZE + STUDY GUIDE (Claude Haiku – Tier 2)
```

```
app.post("/api/study/analyze", async (req, res) => {
```

```
  try {
```

```
    const { userId, subject, quizResults, gradeLevel } = req.body;
```

```
    if (!userId) return res.status(400).json({ error: "userId required" });
```

```
    const correctCount = quizResults.filter(r => r.isCorrect).length;
```

```
    const accuracy = Math.round((correctCount / quizResults.length) * 100);
```

```
    const weakTopics = [...new Set(quizResults.filter(r => !r.isCorrect).map(r =>
```

```
    const strongTopics = [...new Set(quizResults.filter(r => r.isCorrect).map(r =
```

```
const message = await anthropic.messages.create({
```

```
  model: "claude-haiku-4-5-20251001",
```

```
  max_tokens: 4000,
```

```

    messages: [{ role: "user", content: `You are a personalized tutor for ${gra

Student quiz results:
Accuracy: ${accuracy}%
Strong: ${strongTopics.join(", ") || "none"}
Weak: ${weakTopics.join(", ") || "none"}
Details:
${quizResults.map(r => `- ${r.topic} (${r.difficulty}): ${r.isCorrect ? "Correct"

Output ONLY this JSON:
{"performance":{"accuracy":${accuracy},"strong_topics":${JSON.stringify(strongTop

Focus on weak topics. Output ONLY the JSON.` }],
  });

  const raw = message.content.map(b => b.text || "").join("").trim();
  let studyData = null;
  try {
    const clean = raw.replace(/```json|```/gi, "").trim();
    studyData = JSON.parse(clean);
  } catch (e) {
    return res.status(500).json({ error: "Could not parse study guide", raw: ra
  }

  users[userId].quizHistory.push({ date: new Date().toISOString(), subject, acc
  res.json({ studyData, history: users[userId].quizHistory });
} catch (err) {
  res.status(500).json({ error: err.message });
}
});

// SUBSCRIBE
app.post("/api/subscribe", (req, res) => {
  const { userId } = req.body;
  if (!userId) return res.status(400).json({ error: "userId required" });
  if (!users[userId]) users[userId] = { uses: 0, subscribed: false, quizHistory:
  users[userId].subscribed = true;
  res.json({ success: true });
});

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => console.log(`Test Taker API on port ${PORT}`));

```